

Simulation Multivariate Hybrid Data

- 1. Sparsity on v (1-SE)
- 2. Smoothness on v
- 3. Smoothness and Sparsity on v
- 4.1. Sparsity on u (1-SE)
- 4.2. Smoothness on u
- 5. Two-way conditional sparsity
- 6. Two-way conditional smoothness
- 7. Conditional two-way sparsity two-way smoothness
- 8. Multivariate hybrid data case

Multivariate hybrid data case

Let's start with simulating a data with 4 variables (2 functions and 2 regular).

```
set.seed(123)
par(mfrow = c(2, 4), pin = c(1, 1), mar = c(2, 2, 2, 2), pty = "s")

# Common left singular vector u (1 x 50)
x <- seq(0, 2*pi, len = 150)
u <- sin(x); u[1:75] <- 0
#u <- u / sqrt(sum(u^2))

# Variable 1
m1 <- 20
v1 <- rnorm(m1, sd = 0.8)
zero_indices_v1 <- sample(1:m1, 5) # Select 5 indices
v1[zero_indices_v1] <- v1[zero_indices_v1] * 1e-2 + rnorm(5, sd = 0.01)
matplot(v1, type = 'o', main = 'v1, Regular')
abline(h = 0, col = 'red', lty = 2)

X1 <- outer(u, v1) + rnorm(length(outer(u, v1)), sd = 0.03)
image(t(X1), main = 'X1, Regular')

# Variable 2
x2 <- seq(0, 2*pi, length.out = 40)
v2 <- cos(2*x2)
v2[20:40] <- 0

X2 <- outer(u, v2)
X2 <- X2 + rnorm(length(X2), sd = 0.5)
matplot(svd(X2)$v[,1], type = 'l', main = 'v2, Functional')
image(t(X2), main = 'X2, Functional')
```

```

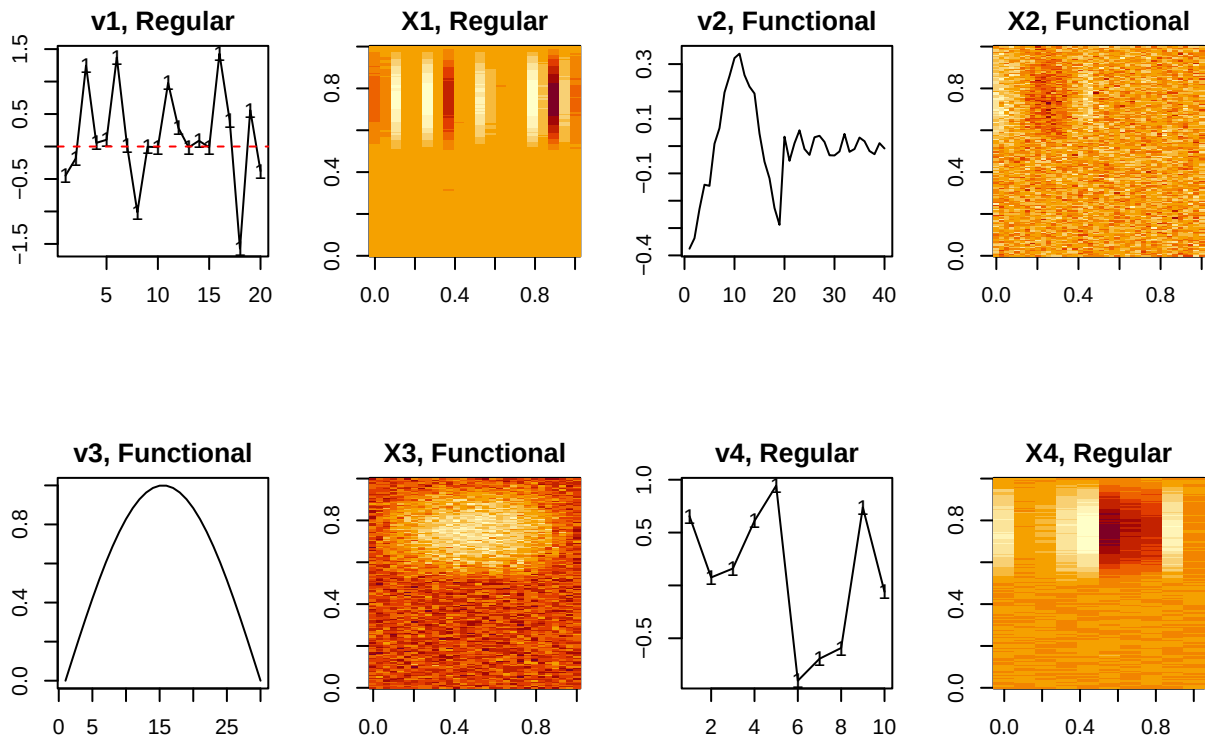
# Variable 3
m3 <- 30
x3 <- seq(0, pi, length.out = m3)
v3 <- sin(x3)
matplot(v3, type = 'l', main = 'v3, Functional')

X3 <- outer(u, v3) + rnorm(length(outer(u, v3)), sd = 0.2)
image(t(X3), main = 'X3, Functional')

# Variable 4
m4 <- 10
v4 <- rnorm(m4)
matplot(v4, type = 'o', main = 'v4, Regular')

X4 <- outer(u, v4) + rnorm(length(outer(u, v4)), sd = 0.05)
image(t(X4), main = 'X4, Regular')

```

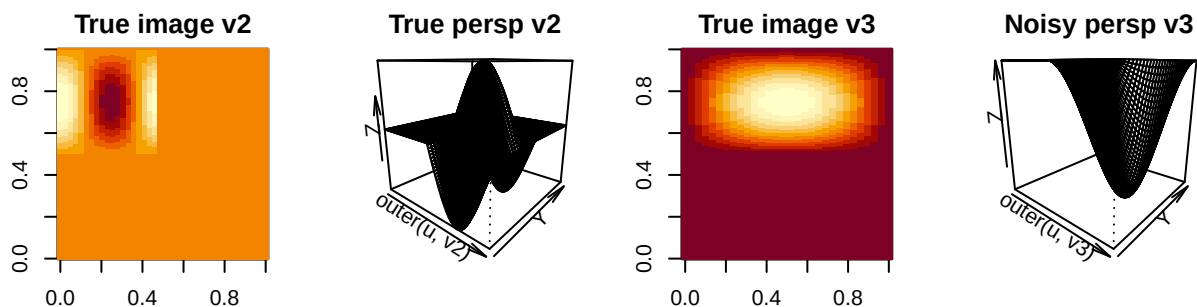


```

# Combine all into clean and noisy hybrid matrices
X <- cbind(outer(u, v1), outer(u, v2), outer(u, v3), outer(u, v4))

# Plot 3D image
par(mfrow = c(1, 4), pin = c(1, 1), mar = c(2, 2, 2, 2))
image(t(outer(u, v2)), main = "True image v2")
persp(outer(u, v2), theta = 40, main = "True persp v2")
image(t(outer(u, v3)), main = "True image v3")
persp(outer(u, v3), theta = 40, main = "Noisy persp v3")

```



Defining objects:

```
rd_object1 <- rdClass(data = as.matrix(X1),
                      Sparsity_parameter = round(seq(1,19, length.out = 10)))

fd_object2 <- fdClass(data = as.matrix(X2),
                      argval = NULL,
                      Smoothing_parameter = NULL,
                      Sparsity_parameter = round(seq(0, 39, length.out = 20)))

fd_object3 <- fdClass(data = as.matrix(X3),
                      argval = NULL,
                      Smoothing_parameter = NULL,
                      Sparsity_parameter = 0) #round(seq(0,29, length.out = 15)))

rd_object4 <- rdClass(data = as.matrix(X4),
                      Sparsity_parameter = 0) #round(seq(0,9, length.out = 5)))

hd_list <- list(rd_object1, fd_object2, fd_object3, rd_object4)
object_list <- hdClass(hdlist = hd_list,
                      argval = NULL,
                      Smoothing_parameter = NULL,
                      Sparsity_parameter = round(seq(0,149, length.out = 20)))

hd <- object_list
n <- nrow(hd); K_fold <- 5; sparse_tuning_type <- "soft"
n_var <- attr(object_list, "n_var") # Number of variables (# of matrices in object_list)
```

```

ncol <- data.frame(attr(object_list, "ncol")) # Number of columns of each matrix

##### Smoothing Parameter #####
# Generate all combinations alphas (one row per combination)
smooth_tuning_col <- expand.grid(attr(object_list, "Smoothing_parameter_col")) # A matrix
smooth_tuning_row <- attr(object_list, "Smoothing_parameter") # A vector

##### level of sparsity (for both functional and non-functional data) #####
sparsity_row_list <- attr(object_list, "Sparsity_parameter")
sparsity_col_list <- attr(object_list, "Sparsity_parameter_col")

X_temp <- hd
# No smoothness list:
S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

cv_sparse_col_SE <- function(data,
                             n_var,
                             ncol,
                             S_alpha_v,
                             S_alpha_u,
                             K_fold,
                             sparse_tuning_result_u,
                             sparse_tuning_result_v,
                             sparse_tuning_type) {

  set.seed(123)
  shuffled_rows <- sample(nrow(data)) # Grouping rows of data matrix
  group_size <- ifelse(round(length(shuffled_rows) / K_fold, digits = 0) == 0,
                      1,
                      round(length(shuffled_rows) / K_fold, digits = 0))

  data_tilde <- data
  fold_errors <- numeric(K_fold) # store errors

  for (k in 1:K_fold) {
    rows_to_remove <- shuffled_rows[((k - 1) * group_size + 1):min(k * group_size, length(shuffled_rows))]
    data_train <- data.frame(data_tilde[-rows_to_remove,])
    data_test <- data.frame(matrix(data_tilde[rows_to_remove,], nrow = length(rows_to_remove)))

    # Power Algorithm
    power_train <- power_algo(data = data_train,
                             n_var = n_var,
                             ncol = ncol,
                             sparse_tuning_result_u = sparse_tuning_result_u,
                             sparse_tuning_result_v = sparse_tuning_result_v,
                             S_alpha_v = S_alpha_v,
                             S_alpha_u = S_alpha_u[-rows_to_remove, -rows_to_remove],
                             sparse_tuning_type = sparse_tuning_type)

    v_train <- power_train[[1]]
  }
}

```

```

u_test <- as.matrix(data_test) %*% as.matrix(v_train)

# Fold-wise error
fold_errors[k] <- sum((data_test - u_test %*% t(v_train))^2) / nrow(data_test)
}
CV_mean <- mean(fold_errors)
CV_se <- sd(fold_errors) / sqrt(K_fold)

return(list(CV_error = CV_mean, SE = CV_se, fold_errors = fold_errors))
}

# Tuning across sparsity levels using 1-SE rule
CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- list()
CV_results <- list()

gamma_v <- rep(0, n_var)
for (i in 1:n_var) {
  gamma_Xi <- sparsity_col_list[[i]]
  cv_means <- cv_ses <- fold_errors_vec <- numeric(length(gamma_Xi))

  for (j in seq_along(gamma_Xi)) {
    gamma_v[i] <- gamma_Xi[j]

    cv_result <- cv_sparse_col_SE(data = X_temp,
                                  n_var = n_var,
                                  ncol = ncol,
                                  S_alpha_v = S_alpha_v,
                                  S_alpha_u = diag(nrow(X_temp)),
                                  K_fold = 10,
                                  sparse_tuning_result_u = 0,
                                  sparse_tuning_result_v = gamma_v,
                                  sparse_tuning_type = sparse_tuning_type)

    cv_means[j] <- cv_result$CV_error
    cv_ses[j] <- cv_result$SE
    #fold_errors_vec <- cv_result$fold_errors
  }

  CV_results[[i]] <- data.frame(gamma_Xi, cv_means, cv_ses)

  # 1-SE rule
  j_min <- which.min(cv_means)
  threshold <- cv_means[j_min] + cv_ses[j_min]
  j_1se <- max(which(cv_means <= threshold)) # Most sparse within 1-SE

  # Store results
  CV_scores_result_v[[i]] <- list(errors = cv_means, SEs = cv_ses)
  sparse_tuning_selection_v <- gamma_Xi[j_1se]
  gamma_v[i] <- sparse_tuning_selection_v
}
gamma_v

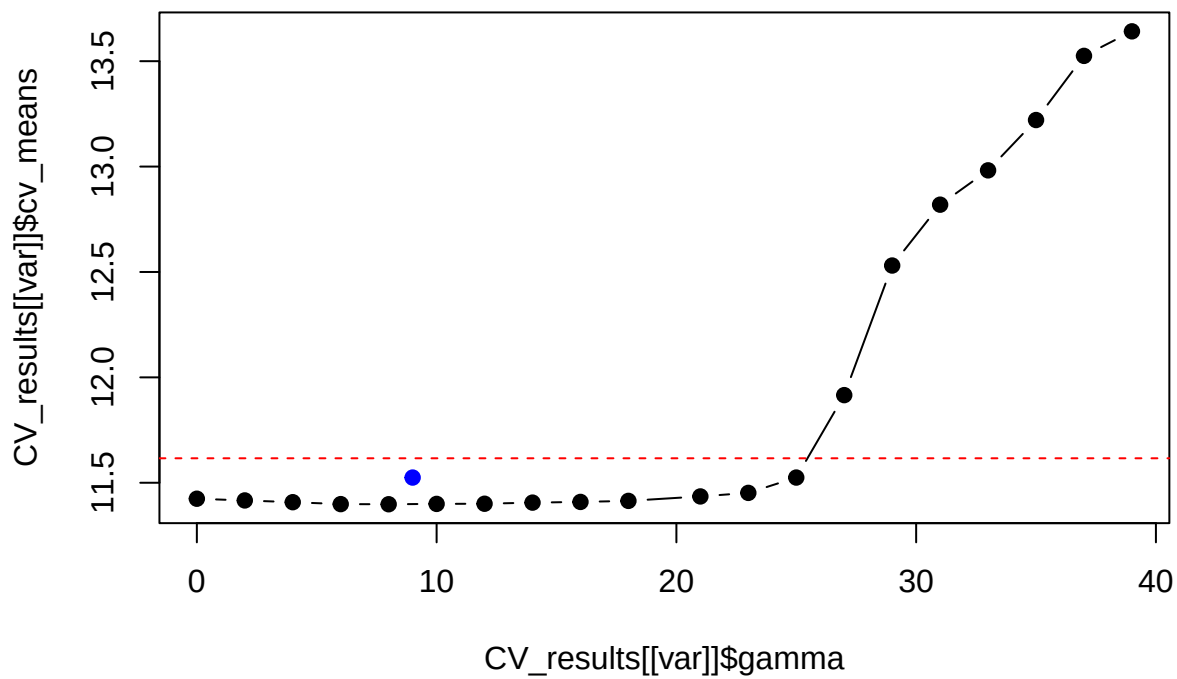
```

```
## [1] 9 25 0 0
```

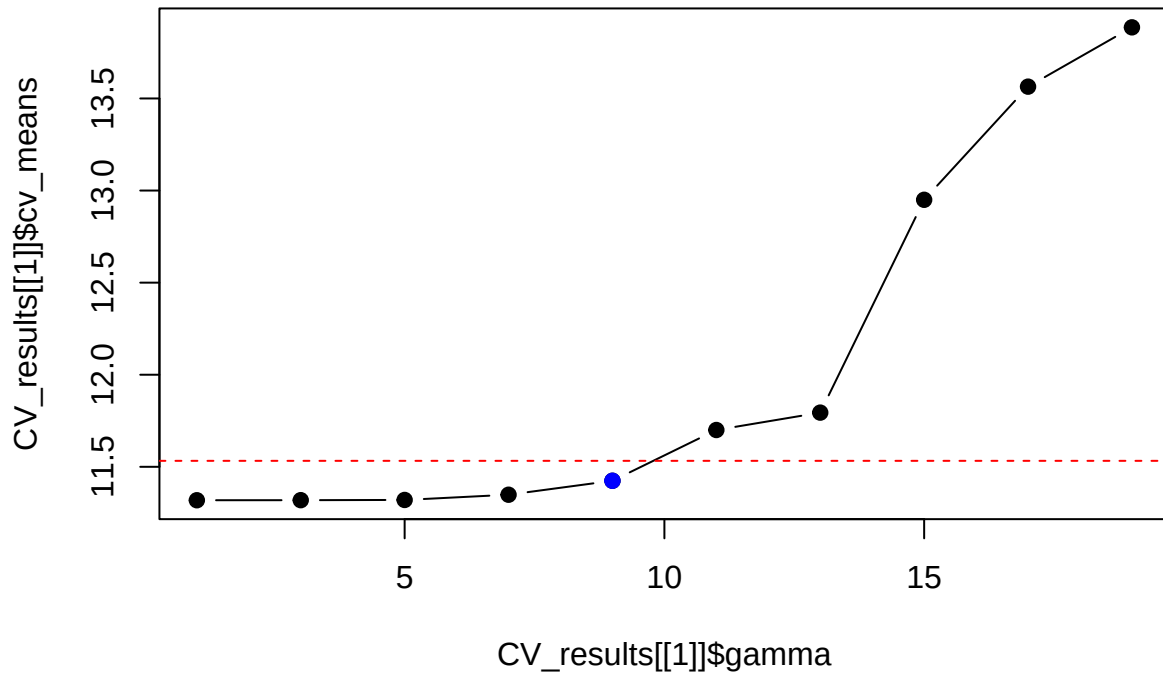
```
var <- 2
CV_results[[var]]
```

```
##      gamma_Xi cv_means    cv_ses
## 1           0 11.42420 0.2116507
## 2           2 11.41606 0.2122263
## 3           4 11.40779 0.2130334
## 4           6 11.39859 0.2160215
## 5           8 11.39804 0.2180092
## 6          10 11.39966 0.2194909
## 7          12 11.40046 0.2205263
## 8          14 11.40573 0.2216311
## 9          16 11.40900 0.2229613
## 10         18 11.41396 0.2245060
## 11         21 11.43512 0.2287529
## 12         23 11.45190 0.2315428
## 13         25 11.52470 0.2393389
## 14         27 11.91573 0.2565817
## 15         29 12.53050 0.3196563
## 16         31 12.81932 0.3223420
## 17         33 12.98211 0.3359498
## 18         35 13.22085 0.3587913
## 19         37 13.52525 0.3859411
## 20         39 13.64158 0.3967402
```

```
plot(CV_results[[var]]$gamma, CV_results[[var]]$cv_means, type = 'b', pch = 19)
abline(h = min(CV_results[[var]]$cv_means) + CV_results[[var]]$cv_ses[which.min(CV_results[[var]]$cv_me
      col = 'red', lty = 2)
points(gamma_v[1], CV_results[[var]]$cv_means[which(CV_results[[var]]$gamma == gamma_v[var])],
      col = 'blue', pch = 19)
```



```
plot(CV_results[[1]]$gamma, CV_results[[1]]$cv_means, type = 'b', pch = 19)
abline(h = min(CV_results[[1]]$cv_means) + CV_results[[1]]$cv_ses[which.min(CV_results[[1]]$cv_means)],
       col = 'red', lty = 2)
points(gamma_v[1], CV_results[[1]]$cv_means[which(CV_results[[1]]$gamma == gamma_v[1])],
       col = 'blue', pch = 19)
```



```

result <- power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = gamma_v, # c(5,22,0,0), # A vector (length = p)
                    S_alpha_v = S_alpha_v, # A list (length = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type = sparse_tuning_type)

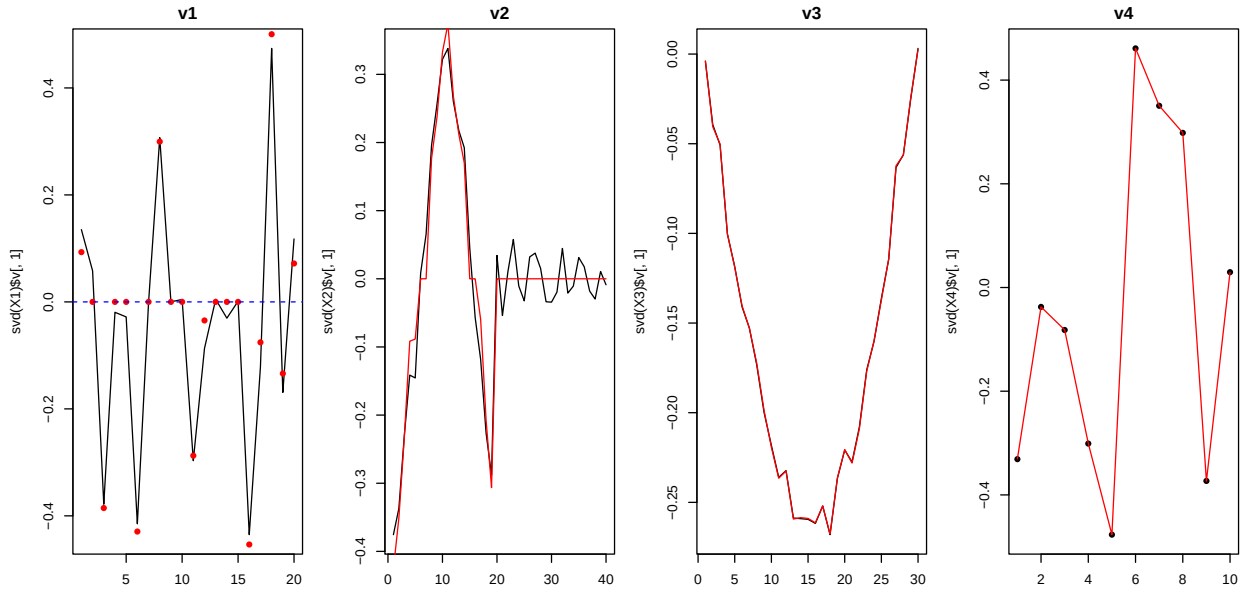
par(mfrow = c(1,4), mar = c(4, 4, 2, 1))
matplot(svd(X1)$v[,1], type = "l", pch = 16, main = "v1")
points(result[[1]][1:20]/norm_vec(result[[1]][1:20]), type = "p", pch = 16, main = "v1", col = 'red')
abline(h = 0, col = 'blue', lty = 2)

matplot(svd(X2)$v[,1], type = 'l', main = "v2")
points(result[[1]][21:60]/norm_vec(result[[1]][21:60]), type = 'l', main = 'v2', col = 'red')

matplot(svd(X3)$v[,1], type = 'l', main = "v3")
points(result[[1]][61:90]/norm_vec(result[[1]][61:90]), type = 'l', main = 'v3', col = 'red')

matplot(svd(X4)$v[,1], type = "p", pch = 16, main = "v4")
points(result[[1]][91:100]/norm_vec(result[[1]][91:100]), type = 'l', main = 'v2', col = 'red')

```

```
# matplot(-result[[1]][1:20], type = "p", pch = 16, main = 'v1')
# abline(h = 0, col = 'blue', lty = 2)
# matplot(-result[[1]][21:60], type = 'l', main = 'v2')
# matplot(-result[[1]][61:90], type = 'l', main = 'v3')
# matplot(-result[[1]][91:100], type = "p", pch = 16, main = 'v4')
```

Let's test smoothness on v:

```
smoothness_type <- "Second_order"
smooth_tuning_col <- expand.grid(attr(object_list, "Smoothing_parameter_col")) # A matrix

##### Grid Points #####
GridPoints_u <- attr(object_list, "GridPoints_u")
GridPoints_v <- attr(object_list, "GridPoints_v")

##### S_alpha for all alphas #####
S_alpha_list_u <- S_alpha_list_v <- Omegas_v <- list()

index <- 0
# S_alpha for v
for (alpha_index in 1:nrow(smooth_tuning_col)) {
  index <- index + 1
  S_u <- S_v <- Omega_v <- Omegas_u <- list()
  for (i in 1:n_var) {
    alpha <- as.numeric(smooth_tuning_col[alpha_index,i])
    if(is.null(GridPoints_v[[i]])){
      S_v[[i]] <- diag(ncol[,i])
      Omega_v[[i]] <- diag(ncol[,i])
    }else{
      get.pen.result <- get.pen(td = GridPoints_v[[i]],
                                alpha = alpha,
                                type = smoothness_type)
      S_v[[i]] <- get.pen.result$S.alpha
      Omega_v[[i]] <- get.pen.result$Omega
    }
  }
}
```

```

    }
  }
  S_alpha_list_v[[index]] <- S_v #as.matrix(bdiag(S_v))
  Omegas_v[[index]] <- Omega_v
}

# S_alpha for u
if(!is.null(smooth_tuning_row)){
  for (alpha_index in 1:length(smooth_tuning_row)) {
    alpha <- as.numeric(smooth_tuning_row[alpha_index])
    get.pen.result <- get.pen(td = GridPoints_u,
                             alpha = alpha,
                             type = smoothness_type)

    S_alpha_list_u[[alpha_index]] <- get.pen.result$S.alpha
    Omegas_u[[alpha_index]] <- get.pen.result$Omega
  }
}

GCV_result_v = opt_alpha_v(X = X_temp,
                           n_var = n_var,
                           ncol = ncol,
                           S_alphas_v = S_alpha_list_v, # A list (length = n_var)
                           S_alphas_u = diag(nrow(X_temp)),
                           alphas_v = smooth_tuning_col,
                           alpha_u = 0,
                           Omega_u = diag(nrow(X_temp)),
                           Omegas_v = Omegas_v,
                           sparse_tuning_result_u = 0,
                           sparse_tuning_result_v = c(0,0,0,0),
                           sparse_tuning_type = sparse_tuning_type)

GCV_result_v$opt.alpha_v

##      Var1          Var2          Var3 Var4
## 55      0 4.485274e-05 0.0006644482      0

#GCV_result_v$GCVdf_v

result <- power_algo(data = X_temp,
                    n_var = n_var,
                    ncol = ncol,
                    sparse_tuning_result_u = 0, # A fixed number
                    sparse_tuning_result_v = gamma_v, # c(5,22,0,0), # A vector (length = p)
                    S_alpha_v = GCV_result_v$opt_s.alpha_v, # A list (length = p)
                    S_alpha_u = diag(nrow(X_temp)), # A matrix
                    sparse_tuning_type = sparse_tuning_type)

par(mfrow = c(1,4), mar = c(4, 4, 2, 1))

# --- Panel 1: v1 ---
matplot(svd(X1)$v[,1], type = "l", pch = 16, main = "v1", lwd = 2)

```

```

points(-v1/norm_vec(v1), type = 'o', col = 'blue')
points(result[[1]][1:20]/norm_vec(result[[1]][1:20]), type = "p", pch = 16, col = 'red', lwd = 2)
abline(h = 0, col = 'green', lty = 2)

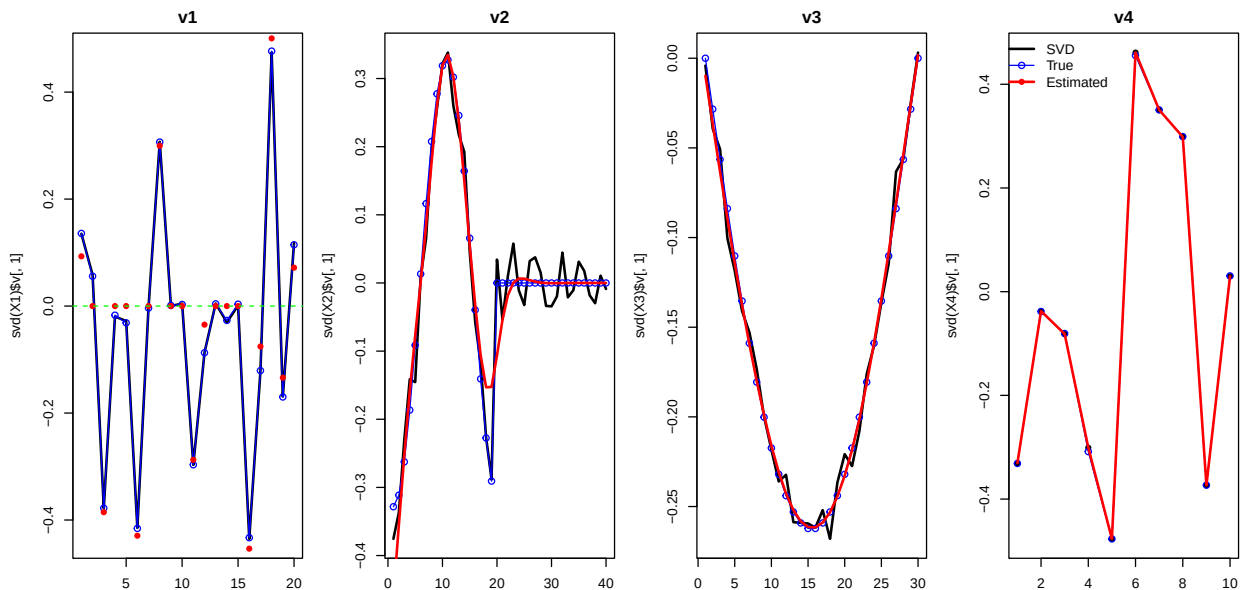
# --- Panel 2: v2 ---
matplot(svd(X2)$v[,1], type = 'l', main = "v2", lwd = 2)
points(-v2/norm_vec(v2), type = 'o', col = 'blue')
points(result[[1]][21:60]/norm_vec(result[[1]][21:60]), type = 'l', col = 'red', lwd = 2)

# --- Panel 3: v3 ---
matplot(svd(X3)$v[,1], type = 'l', main = "v3", lwd = 2)
points(-v3/norm_vec(v3), type = 'o', col = 'blue')
points(result[[1]][61:90]/norm_vec(result[[1]][61:90]), type = 'l', col = 'red', lwd = 2)

# --- Panel 4: v4 ---
matplot(svd(X4)$v[,1], type = "p", pch = 16, main = "v4", lwd = 2)
points(-v4/norm_vec(v4), type = 'o', col = 'blue')
points(result[[1]][91:100]/norm_vec(result[[1]][91:100]), type = 'l', col = 'red', lwd = 2)

# --- Legend ---
par(xpd = TRUE) # Allow legend outside plot
legend(x = 0.5, y = 0.5,
      legend = c("SVD", "True", "Estimated"),
      col = c("black", "blue", "red"),
      lty = c(1, 1, 1),
      pch = c(NA, 1, 16),
      lwd = c(2, 1, 2),
      inset = c(-0.3, 0), # adjust for layout
      bty = "n", horiz = FALSE)

```



```

#matplot(-result[[1]][1:20], type = "p", pch = 16, main = 'v1')
#abline(h = 0, col = 'blue', lty = 2)
#matplot(-result[[1]][21:60], type = 'l', main = 'v2')

```

```
#matplot(-result[[1]][61:90], type = 'l', main = 'v3')
#matplot(-result[[1]][91:100], type = "p", pch = 16, main = 'v4')
```

Finally, let's implement sparsity and smoothness on u and compare the images:

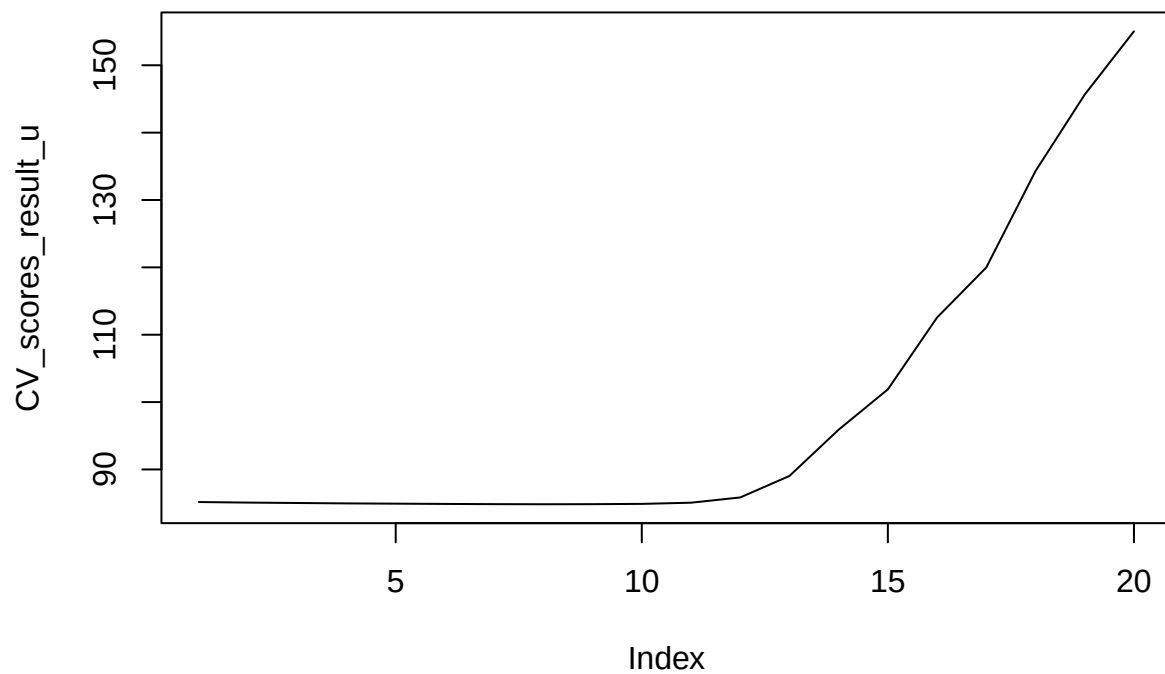
```
sparse_tuning_u <- attr(object_list, "Sparsity_parameter")

S_alpha_v <- list()
for (i in 1:n_var) {
  S_alpha_v[[i]] <- diag(ncol[,i])
}

CV_score_sparse_u <- CV_score_sparse_v <- GCV_score_smooth_u <- GCV_score_smooth_v <- Inf
CV_scores_result_u <- CV_scores_result_v <- c()
result = c()

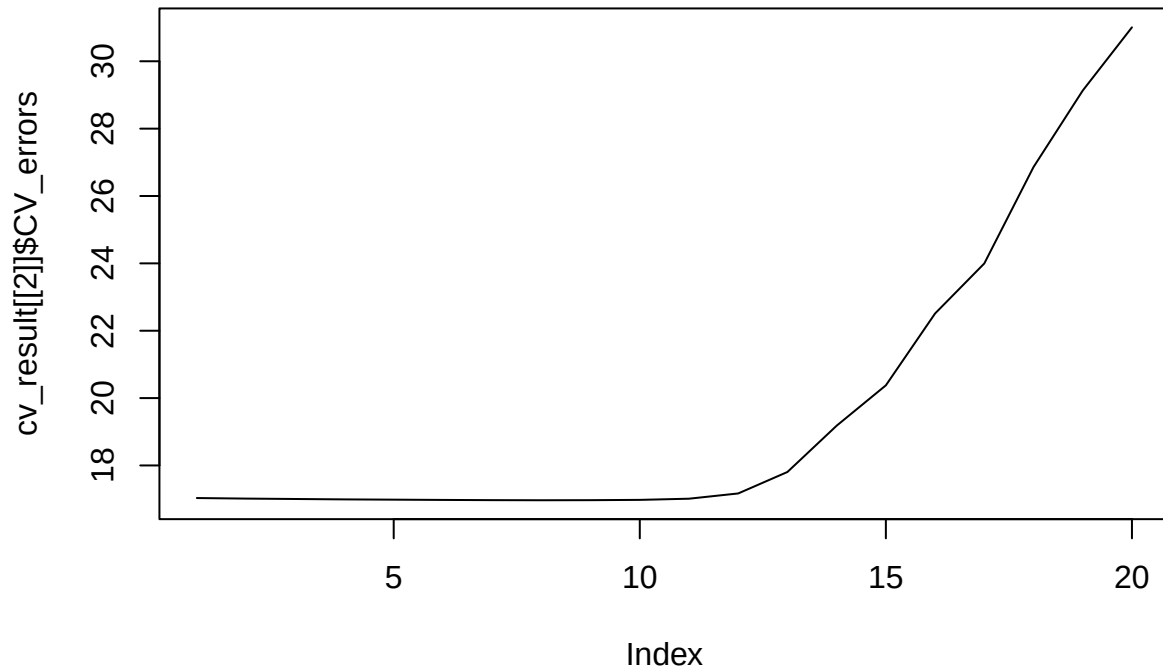
# Step 1: Sparsity on u with no smoothness
for (sparse_tuning_single in sparse_tuning_u) {
  sparse_score = cv_sparse_row(data=X_temp,
                              ncol = ncol,
                              n_var = n_var,
                              S_alpha_v = S_alpha_v,
                              S_alpha_u = diag(nrow(X_temp)),
                              K_fold = K_fold,
                              sparse_tuning_result_u = sparse_tuning_single,
                              sparse_tuning_result_v = rep(0, n_var),
                              sparse_tuning_type = sparse_tuning_type)

  CV_scores_result_u <- c(CV_scores_result_u, sparse_score)
  if (sparse_score <= CV_score_sparse_u) {
    CV_score_sparse_u = sparse_score
    gamma_u = sparse_tuning_single
  }
}
gamma_u1 <- gamma_u
plot(CV_scores_result_u, type = 'l')
```



```
cv_result <- cv_sparse_row_SE(data = X_temp,
                             n_var = n_var,
                             ncol = ncol,
                             S_alpha_u = diag(nrow(X_temp)),
                             K_fold = K_fold,
                             sparse_tuning_result_u = sparsity_row_list,
                             sparse_tuning_result_v = rep(0, n_var),
                             sparse_tuning_type = sparse_tuning_type)

gamma_u2 <- cv_result[[1]]
plot(cv_result[[2]]$CV_errors, type = 'l')
```



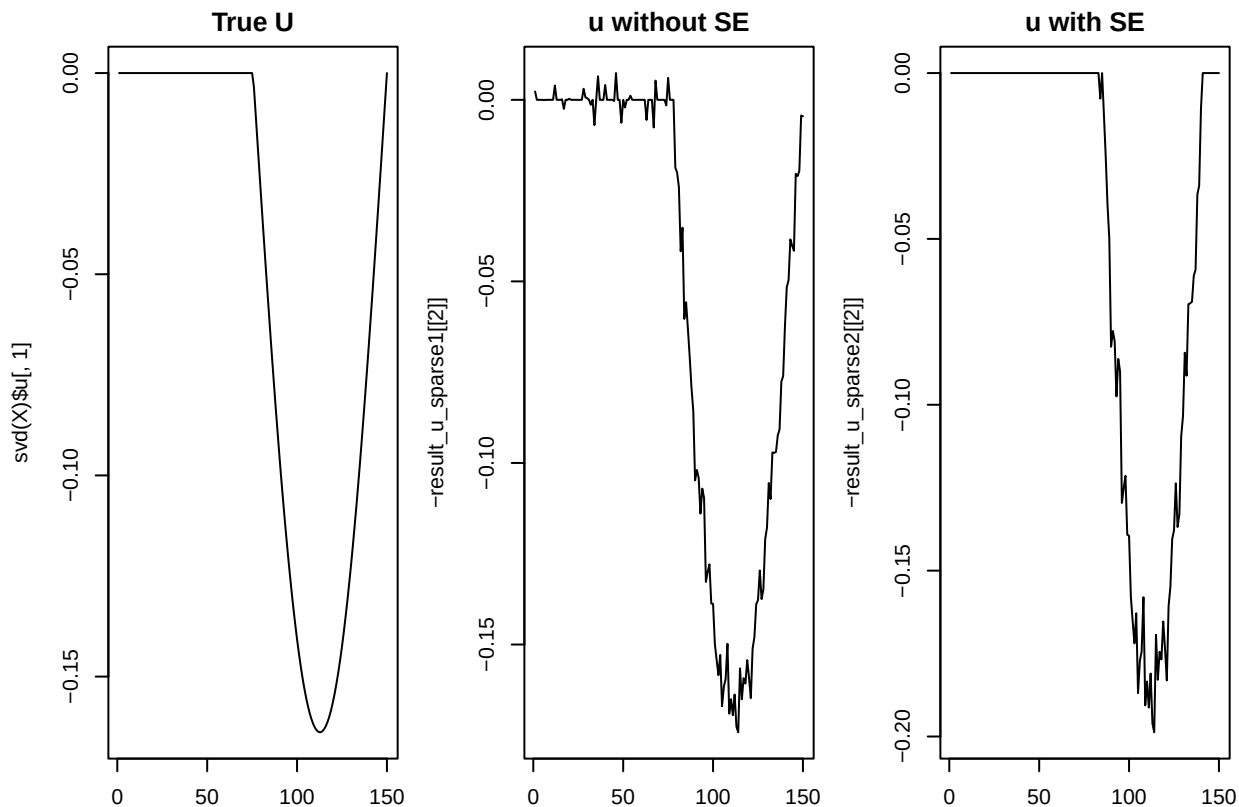
```

result_u_sparse1 <- power_algo(data = X_temp,
                              n_var = n_var,
                              ncol = ncol,
                              sparse_tuning_result_u = gamma_u1, # A fixed number
                              sparse_tuning_result_v = gamma_v, # A vector (length = p)
                              S_alpha_v = S_alpha_v, # A list (length = p)
                              S_alpha_u = diag(nrow(X_temp)), # A matrix
                              sparse_tuning_type = sparse_tuning_type)

result_u_sparse2 <- power_algo(data = X_temp,
                              n_var = n_var,
                              ncol = ncol,
                              sparse_tuning_result_u = gamma_u2, # A fixed number
                              sparse_tuning_result_v = gamma_v, # A vector (length = p)
                              S_alpha_v = S_alpha_v, # A list (length = p)
                              S_alpha_u = diag(nrow(X_temp)), # A matrix
                              sparse_tuning_type = sparse_tuning_type)

par(mfrow = c(1,3), mar = c(4, 4, 2, 1))
matplot(svd(X)$u[,1], type = 'l', main = "True U")
matplot(-result_u_sparse1[[2]], type = 'l', main = "u without SE")
matplot(-result_u_sparse2[[2]], type = 'l', main = "u with SE")

```



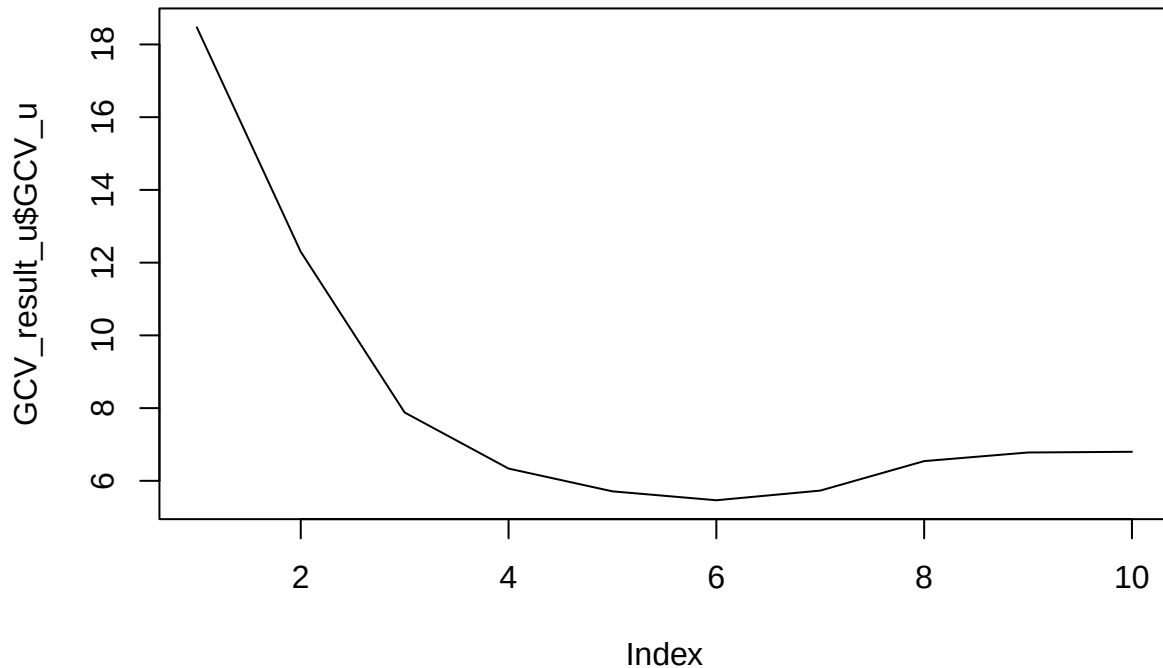
```
ind <- which(apply(smooth_tuning_col, 1, function(row)
  all(abs(row - GCV_result_v$opt.alpha_v) < 1e-12)))

GCV_result_u <- opt_alpha_u(X = X_temp,
  n_var = n_var,
  ncol = ncol,
  S_alphas_v = as.matrix(bdiag(GCV_result_v$opt.s.alpha_v)),
  S_alphas_u = S_alpha_list_u,
  alphas_u = smooth_tuning_row,
  alpha_v = as.numeric(GCV_result_v$opt.alpha_v),
  sparse_tuning_result_u = gamma_u2,
  sparse_tuning_result_v = gamma_v,
  Omega_v = Omegas_v[[ind]],
  Omegas_u = Omegas_u,
  sparse_tuning_type = sparse_tuning_type)

GCV_result_u$opt.alpha

## [1] 0.0006644482

plot(GCV_result_u$GCV_u, type = 'l')
```



```

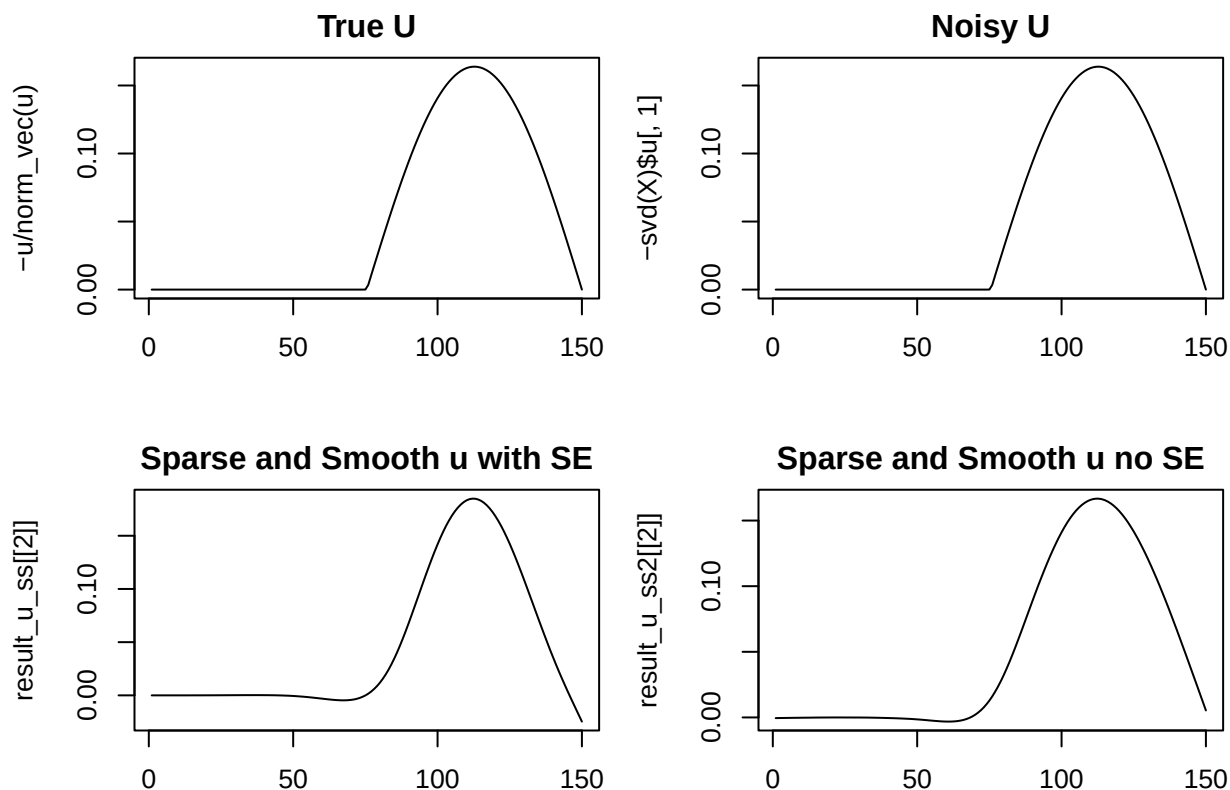
result_u_ss <-power_algo(data = X_temp,
                        n_var = n_var,
                        ncol = ncol,
                        sparse_tuning_result_u = gamma_u2, # A fixed number
                        sparse_tuning_result_v = gamma_v, # A vector (length = p)
                        S_alpha_v = S_alpha_v, # A list (length = p)
                        S_alpha_u = GCV_result_u$opt_s.alpha_u, # A matrix
                        sparse_tuning_type= sparse_tuning_type)

result_u_ss2 <-power_algo(data = X_temp,
                        n_var = n_var,
                        ncol = ncol,
                        sparse_tuning_result_u = gamma_u1, # A fixed number
                        sparse_tuning_result_v = gamma_v, # A vector (length = p)
                        S_alpha_v = S_alpha_v, # A list (length = p)
                        S_alpha_u = GCV_result_u$opt_s.alpha_u, # A matrix
                        sparse_tuning_type= sparse_tuning_type)

par(mfrow = c(2,2), mar = c(4, 4, 2, 1))
u <- sin(x); u[1:75] <- 0
matplot(-u/norm_vec(u), type = 'l', main = "True U")

matplot(-svd(X)$u[,1], type = 'l', main = "Noisy U")
matplot(result_u_ss[[2]], type = 'l', main = "Sparse and Smooth u with SE")
matplot(result_u_ss2[[2]], type = 'l', main = "Sparse and Smooth u no SE")

```

And the final result would be as follows:

```
u_new <- as.vector(result_u_ss[[2]])
v1_new <- as.vector(result_u_ss[[1]][1:20])
v2_new <- as.vector(result_u_ss[[1]][21:60])
v3_new <- as.vector(result_u_ss[[1]][61:90])
v4_new <- as.vector(result_u_ss[[1]][91:100])

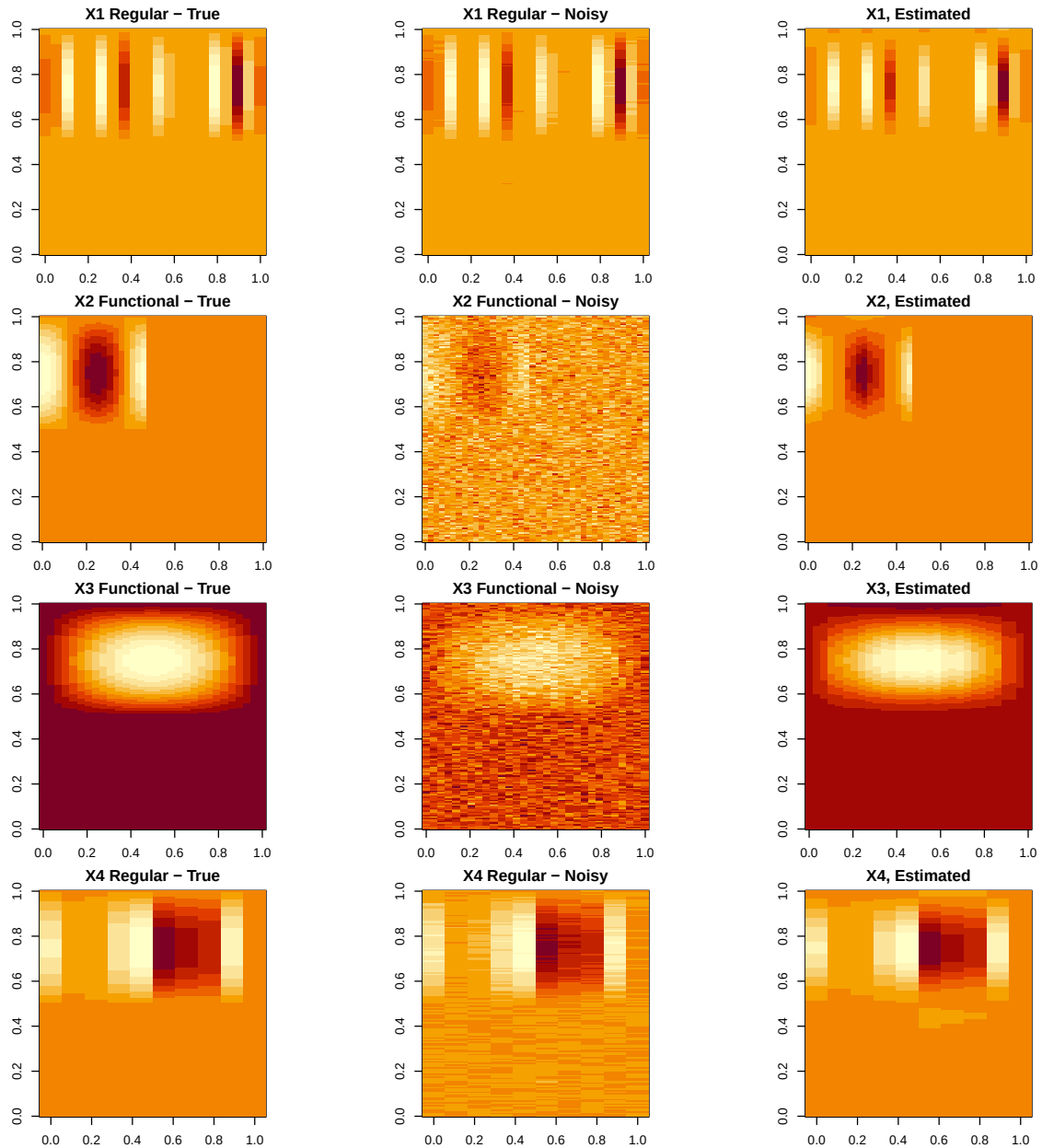
par(mfrow = c(4, 3), pin = c(1, 1), mar = c(2, 2, 2, 2), pty = "s")
# Var1
image(t(outer(u, v1)), main = 'X1 Regular - True')
image(t(X1), main = 'X1 Regular - Noisy')
image(t(outer(u_new, v1_new)), main = 'X1, Estimated')

# Var2
image(t(outer(u, v2)), main = 'X2 Functional - True')
image(t(X2), main = 'X2 Functional - Noisy')
image(t(outer(u_new, v2_new)), main = 'X2, Estimated')

# Var3
image(t(outer(u, v3)), main = 'X3 Functional - True')
image(t(X3), main = 'X3 Functional - Noisy')
image(t(outer(u_new, v3_new)), main = 'X3, Estimated')

# Var4
image(t(outer(u, v4)), main = 'X4 Regular - True')
```

```
image(t(X4), main = 'X4 Regular - Noisy')
image(t(outer(u_new, v4_new)), main = 'X4, Estimated')
```



```
# Plot 3D image
par(mfrow = c(3, 4), pin = c(1, 1), mar = c(2, 2, 2, 2))
# True
image(t(outer(u, v2)), main = "True image v2")
persp(outer(u, v2), theta = 50, main = "True persp v2")
image(t(outer(u, v3)), main = "True image v3")
persp(outer(u, v3), theta = 50, main = "Noisy persp v3")

# Noisy
image(t(X2), main = "Noisy image X2")
persp(X2, theta = 50, main = "Noisy persp X2")
```

```

image(t(X3), main = "Noisy image X3")
persp(X3, theta = 50, main = "Noisy persp X3")

# Reconstructed
image(t(outer(u_new, v2_new)), main = "Reconstructed image v2")
persp(outer(u_new, v2_new), theta = 50, main = "Reconstructed persp v2")
image(t(outer(u_new, v3_new)), main = "Reconstructed image v3")
persp(outer(u_new, v3_new), theta = 50, main = "Reconstructed persp v3")

```

